

Common instructions

1. Cover the entire training syllabus

Your project must demonstrate every topic listed in the training sheet, across MySQL, C#, Basic GUI (HTML/CSS/Bootstrap/JavaScript/jQuery), DevExtreme, Basic API on .NET Framework, and Basic API on .NET Core. A topic you were taught but did not use anywhere is a gap and will be evaluated. *(If a topic does not fit naturally, speak to your reconciler rather than skipping it silently.)* Each project section below maps features to modules to help you, but the mapping is a starting point, not a limit.

2. Role / user-based authentication is compulsory

Every project must implement user login and role-based access control (RBAC). This is mandatory — no project is complete without it. At minimum:

- Users log in with credentials; passwords are stored hashed, never in plain text.
- Use JWT-based/token-based authentication for the APIs.
- Define at least two roles (e.g. normal user and admin/approver). Access to actions and data must differ by role, and this must be enforced on the server — not just hidden in the UI.
- A user must never be able to reach data or actions their role does not permit, even by calling the API directly.

3. Focus on core logic and code quality

Evaluation is weighted heavily toward the database designing, clean coding & logical capabilities. A polished grid over a weak schema will score poorly. Focus your effort on:

- A properly normalized schema with correct primary/foreign keys, indexes, and constraints that enforce your rules at the database level.
- Clean, well-structured APIs. You may build backend on .NET Framework or .NET Core, as per the training sheet.
- Correct use of transactions wherever multiple changes must succeed or fail together.
- Server-side paging, filtering, and sorting — the DevExtreme grid must request data from the API, not load everything and filter in the browser.

4. Not expecting fancy UI

A moderate interface is expected: a DevExtreme DataGrid with working filter / sort / paging (server-side), forms to add and edit data, and at least one summary or chart view. It must exercise your API fully. Do not spend your time on visual polish at the cost of the backend/frontend functionality. Still Mobile friendly UI is appreciated.

5. General rules

- Stay within the taught stack. Do not introduce tools outside the syllabus (for example, use MySQL — not MongoDB).
- The environment may be offline/restricted; do not build anything that depends on external internet services at runtime.
- Write clean, commented code and follow the programming guidelines covered in training.
- Use Swagger to document your APIs and Postman to test them; keep your Postman collection for demo/reference purpose.

- Validate all input on the server. Never trust the client.
- You will demo your project and answer questions on your design choices — be ready to explain why, not just what.

How every project is graded

The same rubric applies to all seven projects so results are comparable.

Area	What we look for
Database design	Normalized schema, correct keys, indexes, constraints that enforce rules, sensible data types
Backend / API	Clean API structure, both .NET Framework & Core, transactions, error handling, security
Query & logic correctness	Correct joins/aggregation, server-side paging/filter/sort, the project's core backend challenge solved
GUI (DevExtreme)	Working grid with server-side ops, forms, one summary/chart — functional, not decorative
GUI (Javascript/jQuery)	Proper use of Object Oriented Programming with sensible industry standards

Note on difficulty: the projects are not identical in difficulty. Your reconciler accounts for this during evaluation. Projects that offer stretch goals let you show extra depth — how far you take them is a signal of your engineering strength.

Project 1: Training & Reconcile Tracker

Turn the batch training sheet into a working web tool.

Objective

Build a tool that manages batches, members, training modules, and topics — tracking work-hours, status, completion dates, and the Jr/Sr reconcile stages for each topic. Reconcilers assign and approve progress stage by stage; a dashboard shows completion percentage per member and per module. Freshers can ask doubts to reconcilers or discuss among other members of the batch.

Core entities (suggested — refine your own schema)

- Batch → Member (a batch has many members)
- Module → Topic (a module has many topics, each with target work-hours)
- Progress record (member × topic: status, hours spent, completed-on, reconcile stage)
- User & Role (trainee, reconciler/admin)

Required features mapped to the training syllabus

Requirement / Feature	Training modules it exercises
Create/manage batches, modules, topics (CRUD)	MySQL DDL/DML; C# CRUD; API on Framework & Core
Track progress and reconcile stages per topic	MySQL DQL, JOINS; transactions
Completion % per member and per module	Aggregate functions (SUM/COUNT), GROUP BY; DevExtreme summaries
Dashboard and reports	DevExtreme DataGrid grouping, charts; server-side paging
Login, trainee vs reconciler roles	JWT auth, authorization (compulsory)

The backend challenge

- Model the module → topic → per-member progress cleanly (master-detail).
- Aggregate hours and completion accurately with GROUP BY across two dimensions.
- Enforce reconcile-stage order so a stage can't be skipped (rule at backend level).

GUI expectation

A grid of modules/topics with per-member progress, a form to update status, and a completion dashboard with summaries. Proper user of Inheritance & other OOP concepts.